

Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 7

по дисциплине «Объектно-ориентированное программирование»

на тему «Разработка приложений с графическим интерфейсом»

Выполнил:
студент группы БИВТ-25-1

Кирюшин А. А.

Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Закрепить навыки разработки программного обеспечения с графическим интерфейсом на языке программирования Си# в проекте Windows Forms.

2. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

В рамках лабораторной работы были решены следующие задачи:

1. Реализовать класс GameLogic для управления состоянием игры
2. Реализовать игровое поле размером 10×10 клеток
3. Хранить состояние игры в двумерном массиве int[10,10]
4. Реализовать условие победы: 5 символов в ряд
5. Реализовать проверку выигрыша в 4 направлениях (горизонталь, вертикаль, 2 диагонали)
6. Реализовать проверку ничьей (заполнено все поле)
7. Реализовать систему ведения счета игроков
8. Реализовать визуализацию выигрышной комбинации (зеленая линия)
9. Реализовать MessageBox с результатом раунда
10. Реализовать кнопки "Новая игра" и "Сбросить счет"
11. Разработать графический интерфейс на Avalonia UI (Canvas)
12. Реализовать обработку кликов мыши (PointerPressed)
13. Реализовать отрисовку сетки, крестиков и ноликов

5. ЛИСТИНГ ПРОГРАММЫ

Класс GameLogic.cs

Класс GameLogic инкапсулирует всю логику игры: состояние игрового поля, управление ходами, проверку условий победы и ничьей, ведение счета игроков.

```
using System;
using System.Collections.Generic;

namespace TicTacToe
{
    public class GameLogic
    {
        private const int BoardSize = 10;
        private const int WinLength = 5;

        private int[,] board; // 0 = пусто, 1 = X
        (игрок 1), 2 = O (игрок 2)
        private int currentPlayer;
        private int player1Score;
        private int player2Score;
        private bool gameOver;

        public int CurrentPlayer => currentPlayer;
        public int Player1Score => player1Score;
        public int Player2Score => player2Score;
        public bool GameOver => gameOver;

        public GameLogic()
        {
            board = new int[BoardSize, BoardSize];
            currentPlayer = 1;
            player1Score = 0;
            player2Score = 0;
            gameOver = false;
        }

        public int GetCell(int row, int col)
        {

```

```

        if (row < 0 || row >= BoardSize || col < 0
|| col >= BoardSize)
            return -1;
        return board[row, col];
    }

    public bool MakeMove(int row, int col)
    {
        if (gameOver || row < 0 || row >= BoardSize
|| col < 0 || col >= BoardSize)
            return false;

        if (board[row, col] != 0)
            return false;

        board[row, col] = currentPlayer;
        return true;
    }

    public void SwitchPlayer()
    {
        currentPlayer = currentPlayer == 1 ? 2 : 1;
    }

    public (bool hasWinner, int winner, List<(int,
int)> winningCells) CheckWin()
    {
        for (int row = 0; row < BoardSize; row++)
        {
            for (int col = 0; col < BoardSize; col+
+)
            {
                int player = board[row, col];
                if (player == 0) continue;

                // Проверка горизонтали (→)
                if (col + WinLength <= BoardSize)
                {
                    bool win = true;

```

```

        for (int i = 1; i < WinLength;
i++)
        {
            if (board[row, col + i] !=
player)

                {
                    win = false;
                    break;
                }
        }
        if (win)
        {
            var cells = new List<(int,
int)>();
            for (int i = 0; i <
WinLength; i++)
                cells.Add((row, col +
i));
            return (true, player,
cells);
        }
    }

    // Проверка вертикали (↓)
    if (row + WinLength <= BoardSize)
    {
        bool win = true;
        for (int i = 1; i < WinLength;
i++)
        {
            if (board[row + i, col] !=
player)

                {
                    win = false;
                    break;
                }
        }
        if (win)
        {

```

```

int)>();
WinLength; i++)
col));
cells);
    }
}

// Проверка диагонали (\)
if (row + WinLength <= BoardSize &&
col + WinLength <= BoardSize)
{
    bool win = true;
    for (int i = 1; i < WinLength;
i++)
    {
        if (board[row + i, col + i]
!= player)
        {
            win = false;
            break;
        }
    }
    if (win)
    {
        var cells = new List<(int,
int)>();
        for (int i = 0; i <
WinLength; i++)
            cells.Add((row + i, col
+ i));
        return (true, player,
cells);
    }
}

// Проверка диагонали (/)

```

```

        if (row + WinLength <= BoardSize &&
col - WinLength + 1 >= 0)
        {
            bool win = true;
            for (int i = 1; i < WinLength;
i++)
            {
                if (board[row + i, col - i]
!= player)
                {
                    win = false;
                    break;
                }
            }
            if (win)
            {
                var cells = new List<(int,
int)>();
                for (int i = 0; i <
WinLength; i++)
                    cells.Add((row + i, col
- i));
                return (true, player,
cells);
            }
        }
    }

    return (false, 0, new List<(int, int)>());
}

public bool CheckDraw()
{
    for (int row = 0; row < BoardSize; row++)
    {
        for (int col = 0; col < BoardSize; col+
+)
        {
            if (board[row, col] == 0)

```

```

        return false;
    }
}
return true;
}

public void SetGameOver()
{
    gameOver = true;
}

public void UpdateScore(int winner)
{
    if (winner == 1)
        player1Score++;
    else if (winner == 2)
        player2Score++;
}

public void ResetGame()
{
    board = new int[BoardSize, BoardSize];
    currentPlayer = 1;
    gameOver = false;
}

public void ResetScore()
{
    player1Score = 0;
    player2Score = 0;
}
}
}

```


Графический интерфейс (MainWindow.xaml)

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/
2006/xaml"
        x:Class="TicTacToe.Views.MainWindow"
        Title="Крестики-нолики 10x10"
        Width="700" Height="800">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" BorderBrush="Gray"
BorderThickness="1"
            Padding="10" Margin="0,0,0,10"
Background="#F0F0F0">
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*/>
                    <ColumnDefinition Width="*/>
                    <ColumnDefinition Width="*/>
                </Grid.ColumnDefinitions>

                <TextBlock Name="Player1ScoreLabel"
Grid.Column="0"
                    Text="Игрок 1 (X): 0"
                    FontSize="16"
                    FontWeight="Bold"
                    Foreground="Blue"
                    VerticalAlignment="Center"/>

                <TextBlock Name="CurrentPlayerLabel"
Grid.Column="1"
                    Text="Ход: Игрок 1"
```

```

        FontSize="16"
        FontWeight="Bold"
        HorizontalAlignment="Center"
        VerticalAlignment="Center"/>

        <TextBlock Name="Player2ScoreLabel"
Grid.Column="2"
        Text="Игрок 2 (0): 0"
        FontSize="16"
        FontWeight="Bold"
        Foreground="Red"
        HorizontalAlignment="Right"
        VerticalAlignment="Center"/>
    </Grid>
</Border>

    <Border Grid.Row="1" BorderBrush="Black"
BorderThickness="2"
        Margin="0,0,0,10">
        <Canvas Name="GameCanvas"
            Width="600" Height="600"
            Background="White"
            PointerPressed="OnCanvasClick"
            ClipToBounds="True"/>
    </Border>

    <StackPanel Grid.Row="2"
Orientation="Horizontal"
        HorizontalAlignment="Center"
Spacing="20">
        <Button Name="NewGameButton"
            Content="Новая игра"
            Click="OnNewGame"
            Width="150" Height="40"
            FontSize="16"/>
        <Button Name="ResetScoreButton"
            Content="Сбросить счет"
            Click="OnResetScore"
            Width="150" Height="40"
            FontSize="16"/>
    </StackPanel>

```

```
        </StackPanel>
    </Grid>
</Window>
```

Логика интерфейса (MainWindow.axaml.cs)

```
onion.Controls.Shapes;
using Avalonia.Input;
using Avalonia.Interactivity;
using Avalonia.Media;
using System;
using System.Collections.Generic;
using System.Linq;

namespace TicTacToe.Views
{
    public partial class MainWindow : Window
    {
        private const int BoardSize = 10;
        private const double CellSize = 60;

        private GameLogic gameLogic;

        public MainWindow()
        {
            InitializeComponent();
            gameLogic = new GameLogic();
            DrawBoard();
            UpdateUI();
        }

        private void DrawBoard()
        {
            GameCanvas.Children.Clear();

            for (int i = 0; i <= BoardSize; i++)
            {
                var vLine = new Line
                {
                    StartPoint = new Avalonia.Point(i *
CellSize, 0),
                    EndPoint = new Avalonia.Point(i *
CellSize, BoardSize * CellSize),
                    Stroke = Brushes.Black,
```

```

        StrokeThickness = 1
    };
    GameCanvas.Children.Add(vLine);

    var hLine = new Line
    {
        StartPoint = new Avalonia.Point(0,
i * CellSize),
        EndPoint = new
Avalonia.Point(BoardSize * CellSize, i * CellSize),
        Stroke = Brushes.Black,
        StrokeThickness = 1
    };
    GameCanvas.Children.Add(hLine);
}

for (int row = 0; row < BoardSize; row++)
{
    for (int col = 0; col < BoardSize; col+
+)
    {
        int cell = gameLogic.GetCell(row,
col);

        if (cell == 1)
        {
            DrawX(row, col);
        }
        else if (cell == 2)
        {
            DrawO(row, col);
        }
    }
}

private void DrawX(int row, int col)
{
    double margin = 10;
    double x = col * CellSize + margin;
    double y = row * CellSize + margin;

```

```

        double size = CellSize - 2 * margin;

        var line1 = new Line
        {
            StartPoint = new Avalonia.Point(x, y),
            EndPoint = new Avalonia.Point(x + size,
y + size),
            Stroke = Brushes.Blue,
            StrokeThickness = 3
        };
        Canvas.SetLeft(line1, 0);
        Canvas.SetTop(line1, 0);
        GameCanvas.Children.Add(line1);

        var line2 = new Line
        {
            StartPoint = new Avalonia.Point(x +
size, y),
            EndPoint = new Avalonia.Point(x, y +
size),
            Stroke = Brushes.Blue,
            StrokeThickness = 3
        };
        Canvas.SetLeft(line2, 0);
        Canvas.SetTop(line2, 0);
        GameCanvas.Children.Add(line2);
    }

    private void DrawO(int row, int col)
    {
        double margin = 10;
        double x = col * CellSize + margin;
        double y = row * CellSize + margin;
        double size = CellSize - 2 * margin;

        var ellipse = new Ellipse
        {
            Width = size,
            Height = size,
            Stroke = Brushes.Red,

```

```

        StrokeThickness = 3,
        Fill = Brushes.Transparent
    };
    Canvas.SetLeft(ellipse, x);
    Canvas.SetTop(ellipse, y);
    GameCanvas.Children.Add(ellipse);
}

private void DrawWinningLine(List<(int, int)>
winningCells)
{
    if (winningCells.Count == 0) return;

    var firstCell = winningCells.First();
    var lastCell = winningCells.Last();

    double x1 = firstCell.Item2 * CellSize +
CellSize / 2;
    double y1 = firstCell.Item1 * CellSize +
CellSize / 2;
    double x2 = lastCell.Item2 * CellSize +
CellSize / 2;
    double y2 = lastCell.Item1 * CellSize +
CellSize / 2;

    var winLine = new Line
    {
        StartPoint = new Avalonia.Point(x1,
y1),

        EndPoint = new Avalonia.Point(x2, y2),
        Stroke = Brushes.Green,
        StrokeThickness = 5
    };
    Canvas.SetLeft(winLine, 0);
    Canvas.SetTop(winLine, 0);
    GameCanvas.Children.Add(winLine);
}

private void OnCanvasClick(object? sender,
PointerPressedEventArgs e)

```

```

    {
        if (gameLogic.GameOver) return;

        var point = e.GetPosition(GameCanvas);
        int col = (int)(point.X / CellSize);
        int row = (int)(point.Y / CellSize);

        if (row < 0 || row >= BoardSize || col < 0
|| col >= BoardSize)
            return;

        if (gameLogic.MakeMove(row, col))
        {
            DrawBoard();
            CheckGameEnd();

            if (!gameLogic.GameOver)
            {
                gameLogic.SwitchPlayer();
                UpdateUI();
            }
        }
    }

    private async void CheckGameEnd()
    {
        var (hasWinner, winner, cells) =
gameLogic.CheckWin();

        if (hasWinner)
        {
            DrawWinningLine(cells);
            gameLogic.SetGameOver();
            gameLogic.UpdateScore(winner);
            UpdateUI();

            string playerName = winner == 1 ?
"Игрок 1 (X)" : "Игрок 2 (O)";
            await ShowMessageBox("Победа!",
"${playerName} победил!");

```



```

    }
    else if (gameLogic.CheckDraw())
    {
        gameLogic.SetGameOver();
        await ShowMessageBox("Ничья!", "Игра
закончилась вничью!");
    }
}

```

```

private async System.Threading.Tasks.Task
ShowMessageBox(string title, string message)
{
    var messageBox = new Window
    {
        Title = title,
        Width = 300,
        Height = 150,
        WindowStartupLocation =
WindowStartupLocation.CenterOwner,
        CanResize = false
    };

    var panel = new StackPanel
    {
        Margin = new Avalonia.Thickness(20),
        Spacing = 20
    };

    panel.Children.Add(new TextBlock
    {
        Text = message,
        FontSize = 16,
        TextWrapping =
Avalonia.Media.TextWrapping.Wrap,
        HorizontalAlignment =
Avalonia.Layout.HorizontalAlignment.Center
    });

    var okButton = new Button
    {

```

```

        Content = "OK",
        Width = 100,
        Height = 35,
        HorizontalAlignment =
Avalonia.Layout.HorizontalAlignment.Center
    };
    okButton.Click += (s, e) =>
messageBox.Close();
    panel.Children.Add(okButton);

    messageBox.Content = panel;
    await messageBox.ShowDialog(this);
}

private void UpdateUI()
{
    Player1ScoreLabel.Text = $"Игрок 1 (X):
{gameLogic.Player1Score}";
    Player2ScoreLabel.Text = $"Игрок 2 (O):
{gameLogic.Player2Score}";

    if (!gameLogic.GameOver)
    {
        string currentPlayerName =
gameLogic.CurrentPlayer == 1 ? "Игрок 1 (X)" : "Игрок 2
(O)";
        CurrentPlayerLabel.Text = $"Ход:
{currentPlayerName}";
    }
    else
    {
        CurrentPlayerLabel.Text = "Игра
окончена";
    }
}

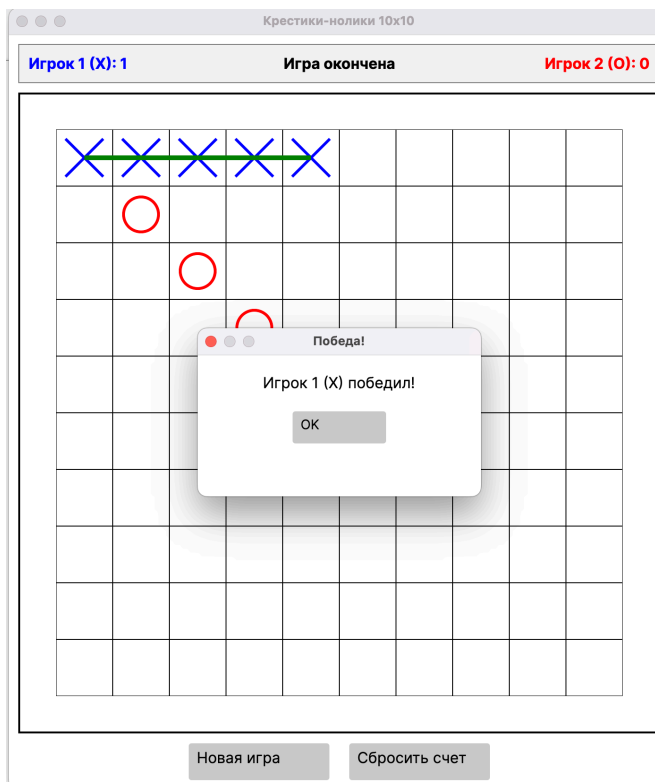
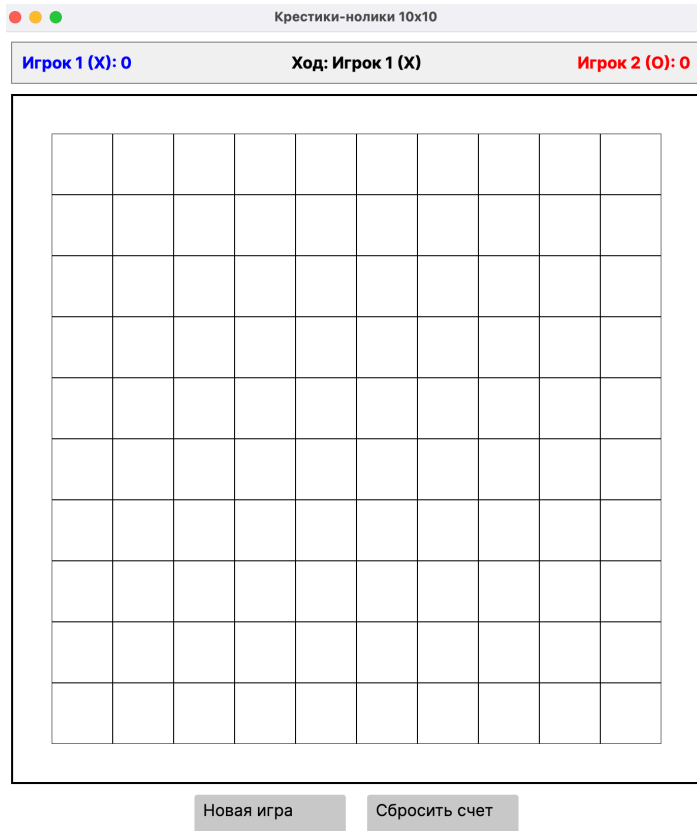
private void OnNewGame(object? sender,
RoutedEventArgs e)
{
    gameLogic.ResetGame();
}

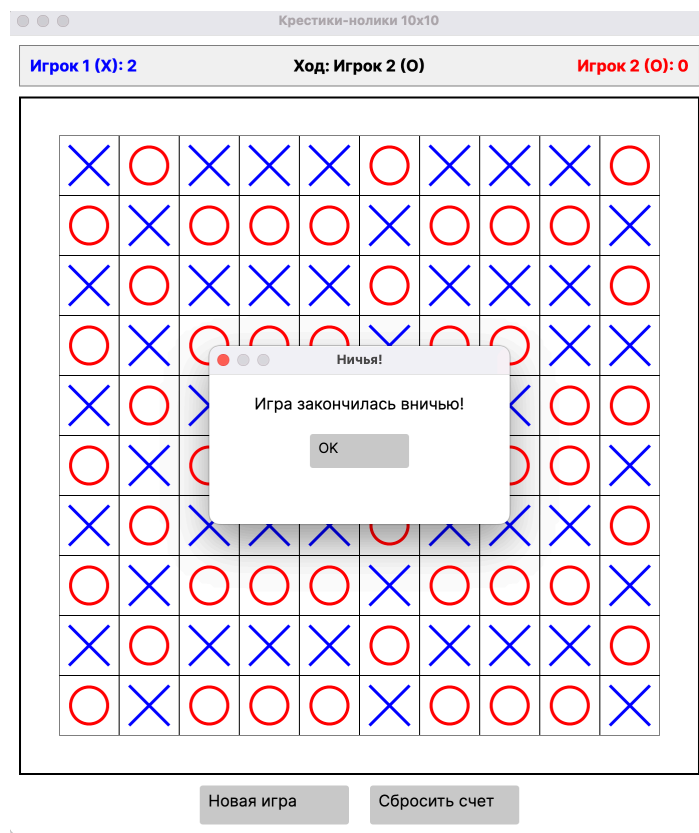
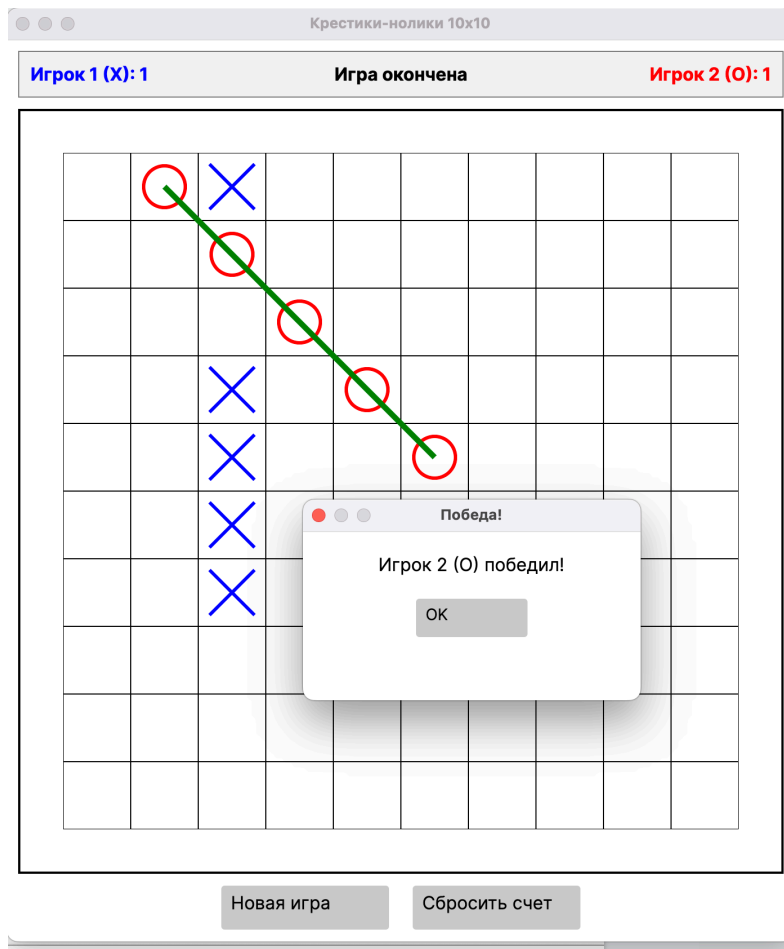
```

```
        DrawBoard();
        UpdateUI();
    }

    private void OnResetScore(object? sender,
RoutedEventArgs e)
    {
        gameLogic.ResetScore();
        gameLogic.ResetGame();
        DrawBoard();
        UpdateUI();
    }
}
```

Выполнение:





7. ВЫВОД

В ходе выполнения лабораторной работы я успешно освоил разработку графических приложений с использованием кроссплатформенного UI-фреймворка Avalonia на языке C#. Получил практический опыт создания интерактивных игровых приложений с полноценным графическим интерфейсом, реализации игровой логики и визуализации состояния через динамическую отрисовку элементов на Canvas. Разработал игру "Крестики-нолики" уровня 2 с расширенными требованиями: поле 10×10 клеток и условием победы 5 символов в ряд, что значительно сложнее классической версии 3×3.

Реализовал класс GameLogic, который полностью инкапсулирует логику игры и хранит состояние в двумерном массиве `int[10,10]`. Разработал эффективный алгоритм проверки победы, который для каждой заполненной клетки проверяет 4 направления: горизонталь, вертикаль и две диагонали. Метод `CheckWin` возвращает кортеж с флагом победы, номером победителя и списком координат выигрышных клеток для отрисовки линии. Освоил работу с кортежами и именованными возвращаемыми значениями, что делает код более читаемым по сравнению с использованием `out`-параметров.

Освоил создание графического интерфейса с использованием XAML-разметки для декларативного описания структуры UI. Разработал трехуровневую структуру: панель счета, Canvas 600×600 пикселей для игрового поля и кнопки управления. Реализовал отрисовку через динамическое добавление визуальных элементов `Line` и `Ellipse` в коллекцию `Canvas.Children`. Метод `DrawBoard` создает сетку 10×10, крестики отрисовываются двумя диагональными синими линиями, нолики - красными кругами `Ellipse`. Освоил позиционирование элементов через `Canvas.SetLeft` и `Canvas.SetTop`.

Реализовал обработку кликов мыши через событие `PointerPressed` с определением клетки по координатам: `col = X / CellSize`, `row = Y / CellSize`. После успешного хода вызывается перерисовка поля, проверка окончания игры, смена игрока и обновление интерфейса. Реализовал визуализацию выигрышной линии через `DrawWinningLine`, которая соединяет центры первой и последней выигрышных клеток зеленой жирной линией. Создал кастомный `MessageBox` через новое окно `Window` с модалным отображением через `await ShowDialog`, применив `async/await` для асинхронных операций UI.

Освоил адаптацию требований `Windows Forms` под `Avalonia UI`. Вместо события `Paint` реализовал декларативную отрисовку через добавление элементов в `Canvas.Children`. Заменял `MouseClick (e.X, e.Y)` на `PointerPressed с e.GetPosition(Canvas)`. Вместо `MessageBox.Show()` создал динамическое окно с кастомным содержимым. Применил принципы ООП: класс `GameLogic` инкапсулирует логику, `MainWindow` отвечает за представление, что обеспечивает слабую связанность компонентов.

Полученные навыки работы с графическими интерфейсами, обработкой событий, динамической отрисовкой и разработкой игровой логики являются фундаментальными для создания интерактивных приложений. Освоил `Avalonia UI` как кроссплатформенный фреймворк для `Windows`, `macOS` и `Linux`. Научился разделять логику и представление, применять XAML для описания UI, работать с

событиями и асинхронными операциями. Эти знания станут основой для разработки более сложных графических приложений и игр на платформе .NET.